Indian Institute of Technology, Hyderabad
Department of Computer Science and Engineering

ID1063: Introduction to Programming
Mid-Semester Examination(Modal Paper)

Roll Number: _____ **Duration: 20 minutes**
Total Marks: 22
Lecture Hall Venue: _____ **Date: 07.10.2025**

Section-Wise Marking Scheme

Section	Type of question	Number of Questions	Marks	Marks Obtained
Section 1	Multiple Choice Questions (MCQ)	1	2	
Section 2	Multiple Select Questions (MSQ)	1	3	
Section 3	Numerical Answer Type (NAT)	1	3	
Section 4	Long Answer Questions	1	6	
Section 5	Coding Questions	1	8	

INSTRUCTIONS PAGE

1. Please fill your roll number correctly and legibly in the space above. Write your full roll number including department code, etc.
2. Do NOT turn over this page till the start of the exam is announced (6:45 pm). Once the exam starts, you may turn over and read the questions overleaf.
3. Please ensure you are carrying only a pen/pencil and your ID card with you.
4. Do NOT carry any electronic devices like mobiles, watches, etc. In case invigilators find any device with you, or any loose sheets/notes, you will be awarded FR, even if you were not intending to use it.
5. Please write your answers in this question paper itself. Please write legibly and within the space provided. Submit the filled-in question paper at the end of the exam to the invigilator (9 pm).
6. In the unlikely situation where you don't have space in the question paper, you may write on a blank sheet and submit it. Do not forget to write your roll number on such a page. Staple it along with this question paper.
7. You may use blank sheets given to you for rough work, which should also be submitted back to us. Staple it along with this question paper. Do not forget to write your roll number on such a page. Unidentified rough sheets found near you may make you a suspect for copying! Please be careful.
8. Any attempt to copy/cheat, etc will straight away lead to an FR grade in the course.
9. After finishing your exam at 9 pm, stop writing. Do not write anything beyond 9 pm. Leave your filled question paper as it is on your desk and move away immediately towards the big screen. The invigilators will collect your sheets and do a tally. After they announce, you can leave the exam hall.

1 Multiple Choice Questions

Note:- Only one of the given options is correct. Circle the correct option among the given options. No partial marking will be given.

[2 points]

1. (2 points) What will be the output of the following code?

```c
#include <stdio.h>
#include <string.h>

int main() {
char s[] = "AzByC";
int n = strlen(s);
int val = 0;

for(int i=0; i<n; i++) {
    if(s[i] >= 'a' && s[i] <= 'z')
        s[i] = s[i] - 32;
    else
        s[i] = (s[i] - 'A' + i) % 26 + 'A';
}

for(int i=0; i<n; i++) {
    if(s[i] == s[n-1-i])
        val += i + 1;
    else if(s[i] < s[n-1-i])
        val -= (s[n-1-i] - s[i]);
    else
        val += (s[i] - s[n-1-i]);
}

printf("%s %d", s, val);
return 0;


}
```

A. AZBZC 5 B. AZBYC 4 C. AZBZC 8 D. BAEZD 7 E. BAEZD 9 F. None of the given options

2 Multiple Select Questions

Note:- One or more than one options may be correct. Circle all the correct option(s) among the given options. Partial marking is given only when all the answered options are correct.

[3 points]

1. (3 points) You are given an integer array `nums` of length n. You are allowed to swap two elements `nums[i]` and `nums[j]` ($i < j$) if and only if gcd(`nums[i]`, `nums[j]`) > 1. The task is to determine whether the array can be rearranged into non-decreasing order using only such swaps. A partial recursive C implementation is provided below, where some conditions have been replaced with blanks. Your job is to select the correct options that should replace the blanks in order for the program to function correctly.

```c
#include <stdio.h>

int gcd(int a, int b);

int canSort(int nums[], int n, int i) {
    if ( /* BLANK 1 */ ) return 1;

    for (int j = i + 1; j < n; j++) {
        if (gcd(nums[i], nums[j]) > 1) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;

            if ( /* BLANK 2 */ ) {
                return 1;
            }
```

```
            temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }
    }
    if ( /* BLANK 3 */ ) return canSort(nums, n, i+1);
    return 0;
}
int main() {
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }
    if (canSort(nums, n, 0)) {
        printf("YES\n");
    } else {
        printf("NO\n");
    }
    return 0;
}
```

Choose the correct option(s) for BLANK 1, BLANK 2, and BLANK 3:

A. BLANK 1: $(i == n - 1)$
B. BLANK 1: $(i == n)$
C. BLANK 1: $(i == 0)$
D. BLANK 1: $(i < 0)$
E. BLANK 2: $canSort(nums, n, i + 1)$
F. BLANK 2: $canSort(nums, n, i)$
G. BLANK 2: $canSort(nums, n, j)$
H. BLANK 2: $canSort(nums, n, n - 1)$
I. BLANK 3: $(nums[i] \leq nums[i + 1])$
J. BLANK 3: $(nums[i] < nums[j])$
K. BLANK 3: $(nums[i] \geq nums[i + 1])$
L. BLANK 3: $(gcd(nums[i], nums[i + 1]) > 1)$
M. All the above mentioned options are correct
N. None of the mentioned options are correct

3 Code correction & Numerical Answer Type Questions

Note 1: First, fix all the syntax errors in the code (if any). Then write the single integer answer in the space provided.

Note 2: While correcting the code, circle the wrong /missing syntax and write the correct syntax at the same place. Marks will be given only if all errors are corrected.

[3 points]

1. (3 points) Consider the following C program:

```
#include <stdio.h>
#include <limits.h>
#define MAX 10
int arr[MAX][MAX];
int split[MAX][MAX];
int main() {
int dims[] = {10,20,30,40};
int n = sizeof(dims) / sizeof(dims[0]);
for (int i = 0; i < MAX i++)
    for (int j = 0; j < MAX; j++)
        arr[i][j] = split[i][j] = -1;
for ( i = 1; i < n; i++)
    arr[i][i] = 0;
for (int len = 2; len < n; len++) {
    for (int i = 1; i <= n - len; i++) {
        int j = i + len - 1;
        arr[i][j] = INT_MAX;
```

```
        for (int k = i; k < j; k++) {
            int cost = arr[i][k] + arr[k + 1][j] + dims[i - 1] * dims[k] * dims[j];
            if (cost < arr[i][j]) {
                arr[i][j] = cost;
                split[i][j] = k;
            }
        }
    }
}
int sumA = 0; sumS = 0;
for (int i = 1; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arr[i][j] != -1)
            sumA += arr[i][j];
        if (split[i][j] != -1)
            sumS += split[i][j];
    }
}
printf("%d\n", sumA + sumS);
return 0;
}
```

---

4 Short Answer Questions

Note: Write the answer in the given space provided. Try to write the answer as short as possible. No partial marking.

[6 points]

1. (6 points) Consider the following C snippet that updates positions in a 2D grid according to a rule:

```
#include <stdio.h>
#define N 5
int main() {
    int grid[N][N] = {0};
    int x = 0, y = 0;
    int vx = 1, vy = 2;
    int steps = 3;
    for(int t = 0; t < steps; t++) {
        grid[y][x] = 1;
        x += vx;
        y += vy;

        if(x >= N) x = N-1;
        if(y >= N) y = N-1;
        if(y < 0) y = 0;
    }
    for(int i = N-1; i >= 0; i--) {
        for(int j = 0; j < N; j++)
            printf("%d ", grid[i][j]);
        printf("\n");
    }
    return 0;
}
```

Answer the following questions using the above C snippet:

(a) (2 points) After the program runs, which cells in the grid will contain '1'? Draw the grid (Just represent the contents of the grid).

.....

.....

.....

.....

(b) (1 point) What is the value of 'y' after the second iteration?

(c) (2 points) Modify the program so that each position is only marked if it has not been marked before, without using any additional arrays. Show the modified code snippet.

(d) (1 point) Suppose 'vx = 2' and 'vy = 1'. Trace the positions marked in the grid for 3 steps.

5 Long Answer Questions

Note: Write the answer in the given space provided. Comment your function properly to clearly specify what your code is doing.

[8 points]

2. (8 points) You are given four integer arrays `A`, `B`, `C`, and `D`, each of size `n`. You need to determine whether there exist elements $a \in A$, $b \in B$, $c \in C$, and $d \in D$ such that:

$$a + b + c + d = 0$$

(a) (4 points) Complete the following C program. It reads four arrays of size `n` from the user and then calls `hasZeroSum()`.

```
#include <stdio.h>
// Function prototype (to be implemented by you)
int hasZeroSum(int A[], int B[], int C[], int D[], int n);
// Assume main function is already implemented and it will print true,
// if the return value of the function is 1 and prints false if the return value of the
    function is 0.
```

(b) (4 points) The program below reads four integer arrays `A`, `B`, `C`, and `D` (each of length `n`). It also computes and stores every sum $a + b$ for $a \in A, b \in B$ into the array `table[]` .

Complete the function `int hasZeroSumUsingTable(int C[], int D[], int nC, int nD, int table[], int tSize)` so that it uses the precomputed values in `table[]` and the elements of `C` and `D` to determine whether there exists a combination of elements from the four arrays that satisfies the required condition. Return 1 if such a combination exists; otherwise, return 0.

Do not use any library search routines — write all logic explicitly in C.

```c
#include <stdio.h>
#define MAX 100

// Function prototype (to be implemented by you)
int hasZeroSumUsingTable(int C[], int D[], int nC, int nD, int table[], int tSize);

int main() {
    int n, A[MAX], B[MAX], C[MAX], D[MAX];
    printf("Enter n: ");
    scanf("%d", &n);

    printf("Enter elements of A: ");
    for (int i=0; i<n; i++) scanf("%d", &A[i]);

    printf("Enter elements of B: ");
    for (int i=0; i<n; i++) scanf("%d", &B[i]);

    printf("Enter elements of C: ");
    for (int i=0; i<n; i++) scanf("%d", &C[i]);

    printf("Enter elements of D: ");
    for (int i=0; i<n; i++) scanf("%d", &D[i]);

    // Build table of all a+b values
    int table[MAX*MAX];
    int tSize = 0;
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            table[tSize++] = A[i] + B[j];
        }
    }

    if (hasZeroSumUsingTable(C, D, n, n, table, tSize))
        printf("There exists a quadruplet with zero sum.\n");
    else
        printf("No such quadruplet exists.\n");

    return 0;
}

// Implement this function to use table[] and elements of C, D
int hasZeroSumUsingTable(int C[], int D[], int nC, int nD, int table[], int tSize) {
    // Write your code here
}
```